

12 — Exercices d'entraînement (modif en direct) — ÉTENDU

18 exercices chronométrés. Fais-les 2-3 fois jusqu'à être fluide. Après chaque modif : rebuild du conteneur concerné + Ctrl+Shift+R. Token de test : voir `04_GUIDE_MODIFICATIONS_DIRECT.md` (section 0).

FRONTEND

F1 — Ajouter une colonne « Email » dans le tableau admin (🕒 5 min)

- **Objectif** : afficher l'email dans la liste des utilisateurs.
- **Fichier** : `Frontend/script/admin/user.js` (`afficherPageUsers`).
- **Solution** : ajouter `<div>Email</div>` dans `headerHTML` (bloc `u-thead`) et `<div class="u-email">${user.email}</div>` dans la ligne `u-row`.
- **Vérif** : recharger `/admin/users` → la colonne apparaît.

F2 — Changer la couleur d'accent d'une page (🕒 4 min)

- **Objectif** : changer la couleur du thème annonces.
- **Fichier** : `Frontend/style/annonceAll.css` (`:root { --ac: ... }`).
- **Solution** : `--ac: 255,120,0;` (orange).
- **Vérif** : recharger `/annonces`, les accents/bordures changent.

F3 — Ajouter un champ « ville » au formulaire d'annonce (🕒 6 min)

- **Objectif** : envoyer la ville avec l'annonce.
- **Fichiers** : la page du formulaire (HTML) + le JS qui construit le `FormData`.
- **Solution** : `<input id="ville">` + `formData.append("ville", document.getElementById("ville").value)`.
- **Vérif** : F12 → Réseau → la requête contient bien `ville`.

F4 — Ajouter un lien dans la barre de navigation (🕒 3 min)

- **Objectif** : ajouter un lien « À propos » dans une nav.
- **Fichier** : une page HTML (bloc `<nav>`).
- **Solution** : `<a class="nav-link" href="/a-propos">À propos</a>` (URL propre, gérée par Nginx).
- **Vérif** : cliquer → la page `/a-propos` s'ouvre.

F5 — Afficher un compteur dynamique (🕒 6 min)

- **Objectif** : afficher le nombre d'annonces trouvées.
- **Fichier** : `Frontend/script/annonces/annonceAll.js` (`displayAnnonces`).
- **Solution** : `document.getElementById("resultCount").textContent = items.length;` (déjà présent → variante : ajouter un `console.log` ou un autre élément).
- **Vérif** : le compteur reflète le filtre.

F6 — Ajouter un bouton de tri (🕒 5 min)

- **Objectif** : trier les annonces par prix décroissant.
- **Fichier** : `Frontend/script/annonces/annonceAll.js` (`sortListings` existe déjà).
- **Solution** : appeler `sortListings('price-desc')` depuis un bouton.
- **Vérif** : l'ordre change à l'écran.

API GO

A1 — Endpoint `GET /admin/users/count` (🕒 8 min)

- **Fichiers** : `API/route/users.go` + `API/admin/users.go` + `API/bdd/userReq.go`.
- **Solution** : code complet dans `04_GUIDE_MODIFICATIONS_DIRECT.md` (section F).
- **Vérif** : `curl http://localhost:8081/admin/users/count -H "Authorization: Bearer $TOKEN" → {"count":N}`.

A2 — Modifier la commission (5 % → 10 %) (🕒 4 min)

- **Fichier** : `API/admin/stripe.go`.
- **Solution** : `(unitAmount * 5)/100 → * 10`, et `0.05 → 0.10`.
- **Vérif** : `cd API && go build ./...` OK, puis `GET /admin/finance/overview`.

A3 — Réserver `/admin/users` aux Admins (🕒 5 min)

- **Fichier** : `API/route/users.go`.
- **Solution** : `auth.VerifyRoleMiddleware(admin.GetAllUsers, "Administrateur")`.
- **Vérif** : token Admin → 200 ; token Salarié → 403.

A4 — Ajouter un champ à une struct + le renvoyer (🕒 8 min)

- **Objectif** : renvoyer le `telephone` de l'utilisateur.
- **Fichiers** : `API/models/users.go` (struct), `API/bdd/userReq.go` (SELECT + Scan).
- **Solution** : ajouter `Telephone string `json:"telephone"` + COALESCE(telephone,")` dans le SELECT + `&user.Telephone`` dans le Scan.
- **Vérif** : `GET /admin/users/1` renvoie le champ `telephone`.

A5 — Endpoint `GET /admin/stats` (🕒 8 min)

- **Objectif** : renvoyer `{users, annonces}`.
- **Fichiers** : `API/route/divers.go` (ou `users.go`) + handler.
- **Solution** : voir `04_GUIDE` section F, exemple 2.
- **Vérif** : `curl http://localhost:8081/admin/stats -H "Authorization: Bearer $TOKEN"`.

A6 — Ajouter une validation d'entrée (🕒 6 min)

- **Objectif** : refuser une annonce sans titre.
- **Fichier** : `API/admin/annonce.go` (`CreateAnnonce`).
- **Solution** :

```
if r.FormValue("titre") == "" {
    http.Error(w, "Le titre est obligatoire", http.StatusBadRequest)
    return
}
```

- **Vérif** : POST sans titre → 400.

BASE DE DONNÉES

D1 — Ajouter une colonne (🕒 5 min)

- **Objectif** : `telephone` sur `utilisateur`.
- **Commande** : `ALTER TABLE utilisateur ADD COLUMN telephone VARCHAR(20) NULL;`
- **Vérif** : `DESCRIBE utilisateur;`

D2 — Requête d'analyse (🕒 4 min)

- **Objectif** : nombre d'annonces par catégorie.
- **Solution** :

```
SELECT c.libelle, COUNT(a.id) nb FROM categorie c
LEFT JOIN annonce a ON a.id_categorie=c.id GROUP BY c.id;
```

- **Vérif** : résultat affiché.

D3 — Modifier une donnée métier (🕒 4 min)

- **Objectif** : valider un utilisateur en attente.
- **Solution** : `UPDATE utilisateur SET validation='Validé' WHERE id=42;`
- **Vérif** : le compte peut se connecter sans être bloqué.

D4 — Export / réimport (🕒 5 min)

- **Objectif** : sauvegarder puis restaurer la base.
- **Solution** :

```
docker exec uc_mysql mysqldump -uupcycle -pupcyclePass123 pa2026 > b.sql
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 pa2026 < b.sql
```

- **Vérif** : `SELECT COUNT(*) FROM utilisateur;` identique.

DOCKER / DEBUG

T1 — Rebuild + logs d'un service (🕒 3 min)

- **Commandes** : `docker compose up -d --build backend` puis `docker compose logs -f backend`.
- **Vérif** : voir « Connexion à la bdd reussie ».

T2 — Diagnostiquer un 500 (🕒 5 min)

- **Méthode** : provoquer une erreur (mauvais param), lire les logs backend, nommer la cause (colonne, scan, nil).
- **Vérif** : savoir expliquer l'erreur.

T3 — Vérifier ports + variables d'env (🕒 3 min)

- **Commandes :**

```
netstat -ano | grep ':8081'  
docker exec uc_backend env | grep -E "DB_|STRIPE|UPLOAD"
```

- **Vérif** : le port 8081 écoute, les variables sont présentes.

Plan d'entraînement conseillé (la veille, 45 min)

1. F1 + F3 (colonne + champ formulaire) — les plus probables.
2. A1 + A2 (endpoint + règle métier) — les plus impressionnants.
3. A3 (protection par rôle) — la question sécurité classique.
4. D1 + T1 (colonne DB + rebuild/logs) — le réflexe complet.